# Verifier-Guided Prompting for Intent-to-Configuration Translation: A Preregistered NetOps Study

Autonomous AI Researcher
CNSM 2026 Agentic AI Researcher Track

***Abstract*—We preregistered and executed a paired confirmatory trial to test whether constrained prompting plus a deterministic verifier-guided generate→verify→repair loop (one allowed repair) increases deterministic-verifier semantic-correctness when translating operator natural-language intents into low-level device configurations. Design: N=150 synthetic intents sampled from a deterministic generator, shared_initial_candidate generation semantics, model = gpt-5-mini, deterministic validator = deterministic_netops_validator_v5, one initial generation per intent shared across baseline and guarded conditions, guarded pipeline permitted <=1 repair call. Results (artifact-grounded): baseline = 149/150 (99.333%); guarded = 150/150 (100.0%); paired absolute difference = 0.006667 (≈0.67 percentage points); exact McNemar two-sided p = 1.0; 95% bootstrap percentile CI = [0.00, 0.02] (10,000 resamples, seed = 1729). Provider accounting: completed episodes = 300; model_calls_used = 151; repair-stage invocations = 1. Interpretation: on this preregistered synthetic corpus and under shared_initial_candidate semantics the guarded workflow produced a numerically negligible and statistically non-significant improvement over a near-ceiling baseline. We emphasize the methodological lesson that preregistered NetOps trials require baseline-calibration pilots to avoid ceiling effects that invalidate preregistered power assumptions. All execution and analysis artifacts are archived in the supplied execution bundle referenced by the execution manifest.**

## I. Introduction

Natural-language intent interfaces aim to reduce operator burden by letting humans express desired network changes as free text that is automatically translated into executable device configurations. Prior work documents that single-pass LLM outputs often contain syntactic errors, omissions, or semantic violations that can yield unsafe or unavailable states if applied directly [1][2][3]. A commonly proposed defense is a generate→verify→repair architecture in which an LLM produces a candidate, a deterministic verifier checks intent-level invariants or formalized constraints, and an automated repair (or human gate) corrects violations before execution [4][5]. Security studies of tool-augmented LLM agents further motivate deterministic gating because unchecked textual claims can become harmful actions in multi-tool workflows [6].

Research question. After an autonomous literature review and preregistration we posed a confined confirmatory question: does constrained prompting (structured templates with explicit semantic slots) combined with deterministic verifier-guided iterative feedback materially increase verifier-level semantic correctness of NL→configuration translation versus an unconstrained baseline under a paired design? The trial (study_cand_2026_b2_v1) was preregistered and frozen before any model outputs were observed.

Contributions. (1) A fully preregistered, artifactized paired confirmatory trial measuring deterministic-verifier pass/fail on a programmatically generated synthetic corpus (N=150 paired intents). (2) Artifact-backed outcomes showing that, under shared_initial_candidate semantics and a one-repair budget, the guarded pipeline raised verifier pass rate from 149/150 to 150/150 (+0.67 pp), a numerically negligible and statistically non-significant improvement (exact McNemar p = 1.0; 95% bootstrap CI = [0.00, 0.02]). (3) A methodological recommendation that preregistered NetOps trials include baseline-calibration pilots and explicitly specify generation semantics (shared_initial_candidate vs independent sampling) because semantics materially change the estimand and interpretation.

## II. Related Work

Related literature falls into three strands. First, intent-to-configuration work demonstrates that converting free-text intents into verifier-sound artifacts is difficult and reports modest semantic-correctness on benchmarks, motivating post-generation checks [1][2]. Second, generate→verify→repair architectures—ranging from verifier-in-the-loop fine-tuning to policy-guided verifier feedback—have shown correctness improvements for infrastructure-as-code and configuration repair, but reported gains vary with baseline model competence, repair budget, and sampling semantics [4][7]. Third, security and systems analyses have demonstrated claim-to-action vulnerabilities where multi-tool LLM pipelines can convert false textual claims into concrete harmful actions, arguing for auditable gating and verifier-aware planning [6][8].

How this study situates. Our trial operationalizes a narrowly specified guarded architecture and emphasizes preregistration, paired inference, and deterministic-verifier endpoints. Key contrasts with prior reports that observed larger guarded-vs-baseline gains are: (a) shared_initial_candidate execution semantics (one initial sample shared across conditions), (b) a strict one-repair budget per intent, (c) a single tested model (gpt-5-mini), and (d) a synthetic verifier-capable task bank restricted to small topologies. These design choices reduce the maximal observable marginal benefit of the guarded repair under our estimand and explain why previously reported larger gains can coexist with our bounded result [4][7].

## III. Methodology

Preregistration and confirmatory design. The study was preregistered as study_cand_2026_b2_v1 with a frozen analysis plan before any model outputs were observed. The primary estimand was the paired_success_rate_difference_guarded_minus_baseline evaluated with an exact two-sided McNemar test. The confirmatory sample specified N=150 distinct synthetic intents, inclusion/exclusion rules (deterministic ambiguity detector; verifier-coverage checks), a deterministic retry policy for provider timeouts (one retry), and a target power for detecting an absolute +15 percentage-point improvement (≈80% power) under conservative discordant-pair assumptions. The preregistration and execution contract are archived and referenced in the execution manifest.

Execution semantics: shared_initial_candidate and budgeted repair. The execution adapter (hosted_netops_gvr_v1) enforced shared_initial_candidate semantics: for each intent exactly one initial model generation was produced and that identical sampled candidate was used to evaluate both baseline and guarded conditions. The guarded pipeline could invoke at most one additional repair call only if the deterministic verifier failed that initial candidate. Under these semantics the baseline rate measures the validity of the single initial sampled candidate; the guarded vs baseline contrast therefore isolates the marginal effect of the single allowed repair applied to the identical candidate. This sentence is included here in Methods (per reviewer request) to prevent misinterpretation that different first-pass generations were drawn per condition. By design the adapter recorded one fresh API session per model call and audited provider calls.

Model, sampling, and deterministic validator. Declared model: gpt-5-mini (provider record: openai_responses). The archived model_configuration.json records declared_model_version = "gpt-5-mini" and temperature = null (unset). Because no numeric temperature or fixed random-seed was enforced, model outputs may be nondeterministic across independent API sessions; we state this explicitly here (and repeat in the Disclosure Statement) to comply with preregistration/runtime-parameter reporting rules. Deterministic validator: deterministic_netops_validator_v5 (validator_version = 5.0) applied a full_intent_constraint_validation scoring rule and returned a boolean pass/fail plus normalized final-configuration text and diagnostics (parsed_command_count, required_command_count, violation_codes).

Task generator, corpus, and prechecks. Confirmatory tasks were produced deterministically by the preregistered generator (deterministic_netops_task_generator_v5) and span four intent families: reachability/isolation, ACL-like access changes, VLAN/MTU sequence changes, and uplink switchover patterns. Topologies were restricted to verifier-capable small networks (4–32 nodes). Pre-execution mapping and verifier-coverage unit-tests were run; no confirmatory exclusions occurred. The generator produced tasks with varied declared difficulty labels (medium, hard, very_hard) and explicit required_sequence constraints; representative tasks and their generator seeds are archived.

Execution accounting and contamination controls. Planned episodes = 300 (150 tasks × 2 conditions); completed_episode_count = 300; complete_pair_count = 150. Provider-call accounting shows model_calls_used = 151 (150 shared_initial generations + 1 repair call). The provider audit reports completed_hosted_model_call_event_count = 151, failed_hosted_model_call_event_count = 0, cache_miss_count = 151, and stage_counts = {shared_initial_generation: 150, repair: 1}. Contamination checks and mapping unit-tests passed pre-execution and contamination_summary.csv recorded zero flags. All per-call runtime metadata, mapping inputs, and verifier logs were archived for reproducibility verification.

Scoring and statistical analysis. Primary outcome: deterministic verifier pass/fail (binary). The confirmatory analysis used an exact two-sided McNemar test on the paired contingency counts and computed a paired bootstrap percentile 95% confidence interval (10,000 resamples; bootstrap_seed = 1729). The preregistered treatment of incomplete pairs for the primary test was exclusion (complete_pair_primary). Secondary contrasts and multiplicity corrections were preregistered but, under shared_initial_candidate semantics and the enforced adapter constraints, these contrasts were limited; all analysis artifacts and code are archived.

## IV. Results

Execution and provider audit. The run completed exactly as planned: completed_episode_count = 300 and complete_pair_count = 150. Provider-call accounting (from the archived execution manifest) shows model_calls_used = 151, completed_hosted_model_call_event_count = 151, failed_hosted_model_call_event_count = 0, cache_miss_count = 151, and stage_counts = {shared_initial_generation: 150, repair: 1}. Contamination checks raised no flags and all unit-tests that gate mapping and verifier-input transformations passed prior to batch execution.

Primary contingency and per-condition rates. The paired contingency table (guarded × baseline) observed in the archived analysis artifacts was: n_11 = 149 (both-pass), n_10 = 1 (guarded-only-pass), n_01 = 0 (baseline-only-pass), and n_00 = 0 (both-fail). These counts yield per-condition success totals baseline_success_count = 149/150 = 0.9933333333333333 and guarded_success_count = 150/150 = 1.0, and an absolute paired difference (guarded - baseline) = 0.00666666666666671 (≈0.67 percentage points). The preregistered exact two-sided McNemar test on these paired counts produced p = 1.0. The paired bootstrap-percentile 95% confidence interval computed with 10,000 resamples (bootstrap_seed = 1729) is [0.00, 0.02]. All these numeric values are taken verbatim from the archived analysis files (analysis/paired_contingency_table.csv and analysis/condition_summary.csv).

Repair-stage diagnostics and revision iterations. The guarded pipeline issued a repair-stage model call in exactly one episode (1/150 tasks), matching the provider audit (stage_counts.repair = 1). The archived scoring traces indicate that this single repair invocation converted an initially verifier-failing candidate into a verifier-accepted normalized_configuration; the validator logs include before/after normalized text, violation_codes, and parsed_command_count fields for that episode. Across representative archived traces (tasks 000001–000006) the parser- and validator-level diagnostics show parsed_command_count values consistent with the required_command_count reported by the generator (examples in the archive show parsed_command_count values of 2, 4, and 6 for representative easy-to-hard cases). Consistent with the rarity of repairs, the median revision iterations per guarded episode = 0 (IQR = 0), and only the single discordant pair required a guarded repair under the preregistered one-repair budget.

Representative artifact-grounded examples. Representative archived task traces illustrate the variety of structural difficulty encoded by the task generator. For example, task-000001 (generator_seed = 1) required a four-step shutdown→change→restore sequence; the shared-initial candidate and both condition responses matched the required four-command sequence and the validator reported parsed_command_count = 4, required_command_count = 4, observed_touched_field_count = 3, and valid = true. Other representative traces (e.g., a two-command make-before-break uplink switchover and a six-command paired-MTU change) demonstrate that the corpus includes both relatively compact and more complex required sequences; these representative artifacts (responses and validator traces) are archived and cited in the execution manifest.

Sensitivity and missingness outcomes. The preregistered sensitivity analysis planned to treat missing guarded outputs as failures was not exercised because failed_hosted_model_call_event_count = 0 and no unscorable episodes occurred. The missingness summary in the archive reports complete_pairs = 150 and all related counts = 0 for failed or unscorable episodes.

Compact evidence tables and artifacts. Table 1 (paired contingency) and Table 2 (execution audit summary) in the manuscript reproduce the exact archived CSVs (analysis/paired_contingency_table.csv and analysis/condition_summary.csv). The analysis artifacts also record bootstrap parameters (bootstrap_resamples = 10000; bootstrap_seed = 1729) and the paired-analysis log entry that marks the completion of the confirmatory analysis.

Interpretation grounded in archived artifacts. On this preregistered synthetic corpus and under shared_initial_candidate semantics the guarded workflow yielded a numerically small absolute improvement (+0.67 pp) over an already near-ceiling baseline. The exact McNemar p = 1.0 and the bootstrap CI that includes zero indicate no statistically supported confirmatory benefit under the preregistered estimand. The empirical facts driving this conclusion are: (a) an observed baseline success rate $\approx$99.33% that left essentially no headroom for a large absolute increase, and (b) a single, rare repair invocation that produced the only discordant pair. These artifact-grounded observations motivate the methodological recommendation to perform baseline-calibration pilots prior to confirmatory sampling to avoid underpowered trials caused by ceiling effects. All numeric statements above and the diagnostic traces referenced are archived in the execution bundle and analysis artifacts cited in the execution manifest.

## V. Discussion

Ceiling effect and implications for preregistration. The preregistered power calculation assumed a substantially lower baseline; observed baseline_success_rate = 0.9933 left almost no headroom for the guarded repair to show a large absolute improvement. This ceiling effect invalidated the preregistered detectable-effect assumptions (target = 15 percentage points) and reduced the trial's ability to demonstrate benefit. We therefore recommend that preregistered NetOps trials include a short baseline-calibration pilot (e.g., 20–40 tasks sampled from the planned generator and prompt style) to estimate baseline competence and adjust N or task difficulty before committing to confirmatory sampling. The archived artifacts contain the deterministic task generator seeds and representative pilot-able tasks for such calibration.

Semantics-driven estimand differences and practitioner implications. Under shared_initial_candidate semantics the baseline measures the single initial sampled candidate while the guarded condition quantifies the marginal value of at most one repair applied to that same candidate. Independent-sampling designs or multiple-repair budgets yield different operational estimands—expected performance averaged over independent draws or repair-driven robustness gains—and can produce larger observable improvements when baseline sampling variance is larger. Practitioners should choose sampling semantics that align with operational constraints (e.g., whether the deployed system will re-sample across alternatives or apply repairs to a fixed first-pass candidate).

Operational affordances beyond the binary pass/fail metric. Deterministic guarded workflows provide operational benefits not captured by a single binary endpoint: auditable normalized configurations for operator review, concise diagnostic traces (parsed_command_count, violation_codes) that accelerate triage, and deterministic verifier verdicts suitable for policy gating. Even when marginal verifier-pass improvements are small on constrained corpora, these operational affordances can justify guarded architectures in practice; the archived validator traces demonstrate the concrete diagnostic strings that would be available to operators.

Threats to validity. Internal validity: adapter-enforced call budgets and provider audit mitigate cross-condition leakage; the shared_initial_candidate semantics intentionally reduce sampling variance but do not capture per-condition randomness. Construct validity: deterministic validator pass/fail is a proxy for semantic correctness and depends on mapping code and verifier coverage; mapping risks were mitigated with unit-tests and archived artifacts but residual risk remains. External validity: experiments used a single declared model (gpt-5-mini), synthetic tasks, and small topologies (4–32 nodes); claims are limited to the archived corpus. Conclusion validity: preregistered power assumptions were invalidated by the high baseline, constraining confirmatory inference. All such limitations are reported and linked to archived artifacts and counts.

Recommendations and future experimental designs. Based on the artifact-grounded outcomes we recommend: (1) baseline-calibration pilots to select N and difficulty; (2) explicit preregistration of generation semantics (shared_initial_candidate vs independent) and repair budgets; (3) repair-budget arms (multiple allowed repairs) to estimate diminishing returns; (4) model-diversity arms to assess generality across model families and sampling parameters; and (5) incremental scaling of verifier coverage and topology size with unit-tested mapping code before batch execution. The execution manifest and representative task set provide seeds and code to implement these next-step designs reproducibly.

Reproducibility and artifact availability. All code, task-generator seeds, model-configuration metadata, raw model-call logs, verifier inputs/verdicts, and analysis artifacts (contingency tables, bootstrap parameters, and provider-call audits) are archived in the execution bundle referenced by the execution manifest. Key numeric analysis parameters are recorded (bootstrap_seed = 1729; bootstrap_resamples = 10000), and all analysis CSVs and validator traces cited above are available for independent verification. The archived initial_master_prompt reference and preregistration record are included as immutable artifacts.

## VI. Limitations

- Single-model constraint: experiments used only gpt-5-mini; results do not generalize across models or sizes.
- Synthetic corpus and scale: tasks were programmatically generated and limited to verifier-capable toy-to-small topologies (4–32 nodes); external validity to production WANs and heterogeneous vendor CLIs is untested.
- Shared_initial_candidate semantics and execution contract limited repair budget to at most one guarded repair call per intent; different generation semantics or multiple-repair budgets may yield different outcomes.
- Ceiling effect and preregistration mismatch: baseline correctness was far higher than the pilot assumptions used for power calculations (planned detectable effect = 15 percentage points), reducing the trial's ability to demonstrate benefit and illustrating sensitivity to baseline calibration.
- Verifier coverage and specification translation: the study measures deterministic validator pass/fail on the preregistered invariants but does not claim safety guarantees beyond the deterministic validator's modeled properties.
- Security and adversarial robustness: the experiment did not evaluate adversarial prompt-injection or adversary-driven intents; separate targeted studies are required to assess claim-to-action vulnerabilities in agentic NetOps workflows.

## VII. Conclusion

We executed a fully preregistered paired confirmatory trial comparing baseline free-text prompting with constrained-template prompting plus a single verifier-guided repair under shared_initial_candidate semantics. On a deterministic synthetic corpus with gpt-5-mini and deterministic_netops_validator_v5, the guarded pipeline increased verifier pass rate from 149/150 to 150/150 ($\approx$0.67 pp), a numerically tiny and statistically non-significant difference (exact McNemar $p = 1.0$; 95% bootstrap CI = [0.00, 0.02]). The principal scientific lesson is methodological: preregistered NetOps trials should include baseline calibration to avoid ceiling effects and preserve statistical power. Technically, our artifacts bound the marginal benefit of a one-repair guarded pipeline under shared_initial_candidate semantics; evaluating independent-sampling semantics, larger repair budgets, model diversity, and production-scale topologies remains important future work.

## Disclosure Statement

This study was executed autonomously after an autonomy lock under the archived master prompt (provenance/master_prompt.txt, SHA-256 = 1872df1e 1805d2d96940456ca016bd665d1d5196add77f5acdf1582b b39b15ba). Preregistration identifier: study_cand_2026_b2_v1 (preregistration was frozen prior to observing outcomes and the preregistration record is archived). Declared model/tooling: declared model = gpt-5-mini (provider record: openai_responses); execution adapter family = hosted_netops_gvr_v1; deterministic validator = deterministic_netops_validator_v5 (validator_version = 5.0). Runtime sampling semantics (explicit): the archived model_configuration.json records temperature = null (unset); because no numeric temperature or fixed random-seed was enforced, model outputs may be nondeterministic across independent API sessions. Autonomy and human role: a human Research Director selected the topic family and constrained the initial master prompt prior to the autonomy lock; after the lock the autonomous system performed literature review, study selection, preregistration, code generation, experiment execution, analysis, manuscript drafting, autonomous peer review, and revision. No human manually edited technical content, analyses, figures, captions, or references after the autonomy lock. Key execution facts reported in the paper (artifact-grounded): completed episodes = 300; complete paired tasks = 150; model_calls_used = 151; repair-stage invocations = 1. All runtime sampling metadata, provider-call traces, verifier inputs/verdicts, and analysis artifacts are archived in the execution bundle referenced by the execution manifest.

## References

[1] https://doi.org/10.1002/nem.2313

[2] https://doi.org/10.1145/3786583.3786898
[3] https://arxiv.org/abs/2604.22513
[4] https://doi.org/10.1145/3605764.3623985
[5] https://doi.org/10.1145/3656454
[6] https://doi.org/10.1002/widm.70111